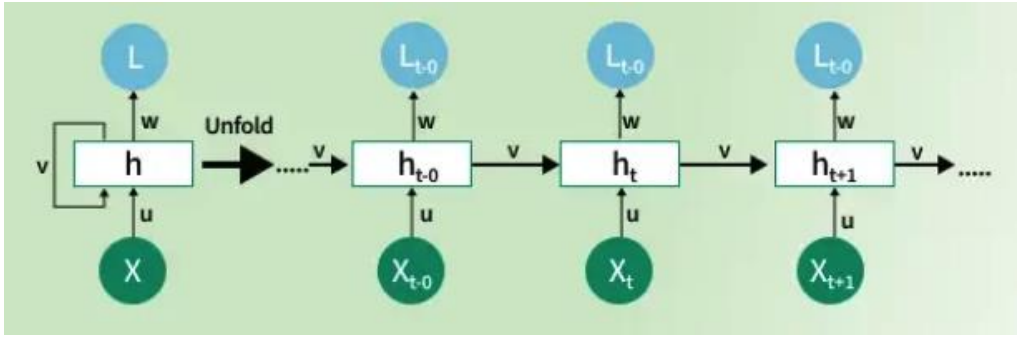
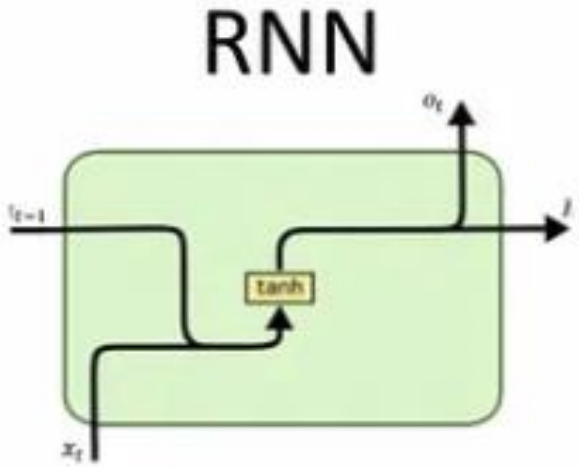
Implementing Recurent Neural Networks

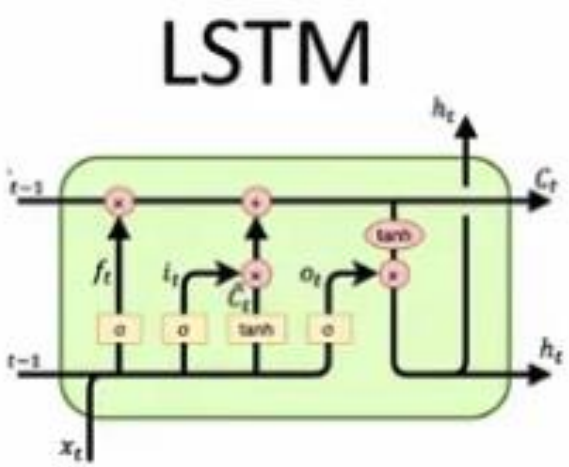
- Architecture
- Training
- Improvements
- Results: what to expect?



Languague Models

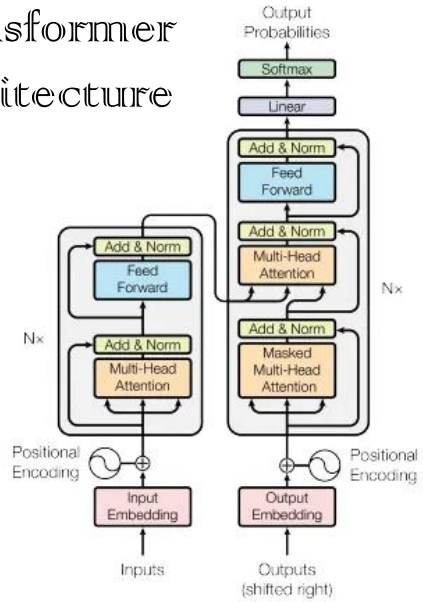


Simplest neural networks that can deal with sequential data



A special class of RNNs that can have a longer short-term memory compared to vanilla RNNs

Transformer architecture

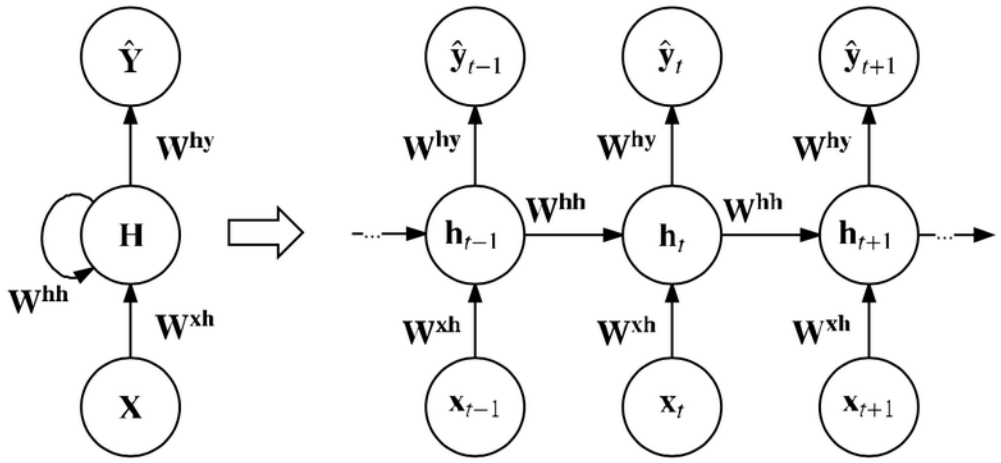


Advanced model that is the engine behind ChatGPT and other LLMs

We are going to study these 3 very important architectures, quite relevant in the development of Large Languague Models

https://medium.com/@adria.cabello/the-evolution-of-language-models-a-journey-through-time-3179f72ae7eb

RNNS: the idea of recurrent neural networks is quite old, as together with the feedforward architecture are the basic ones. One very important reference is the fundational Backpropagation algorithm paper, where the potential of RNN for learning sequences is analyzed.



Learning Internal Representations by Error Propagation

D. E. RUMELHART, G. E. HINTON, and R. J. WILLIAMS

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning internal representations by error propagation. In D. E. Rumelhart & J. L. McClelland (Eds.), Parallel Distributed Processing (Vol. 1, pp. 318–362). MIT Press.

Relevant works related to Recurrent Neural Networks

1986; Hopfield Model of Associative Memories using RNN.. Pioneering work in ANN.

1990: The Backpropagation trhought time (BPTT) algorithm first proposed by Werbos.

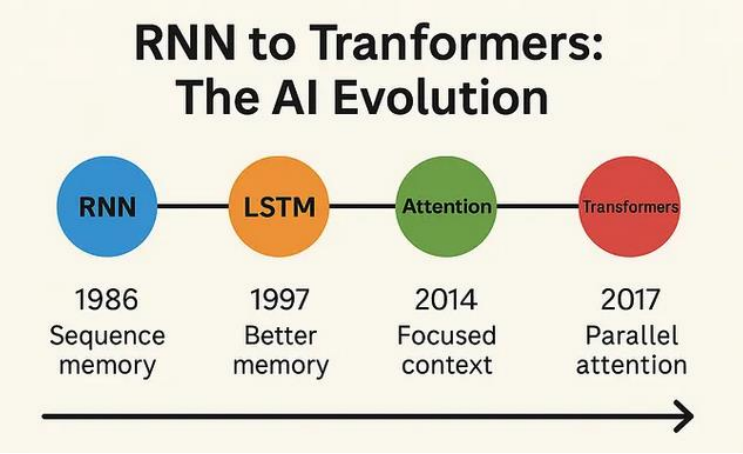
1990: The Elman network introduced in 1990. The first succesful RNN trained with BackPropagation Trhough Time applied to sequences and language processing.

1997: Long Short-Term Memory. Hochreiter, S. & Schmidhuber, Jürgen.

2014: Gated Recurrent Unit Model (GRU) Cho, K., et al.

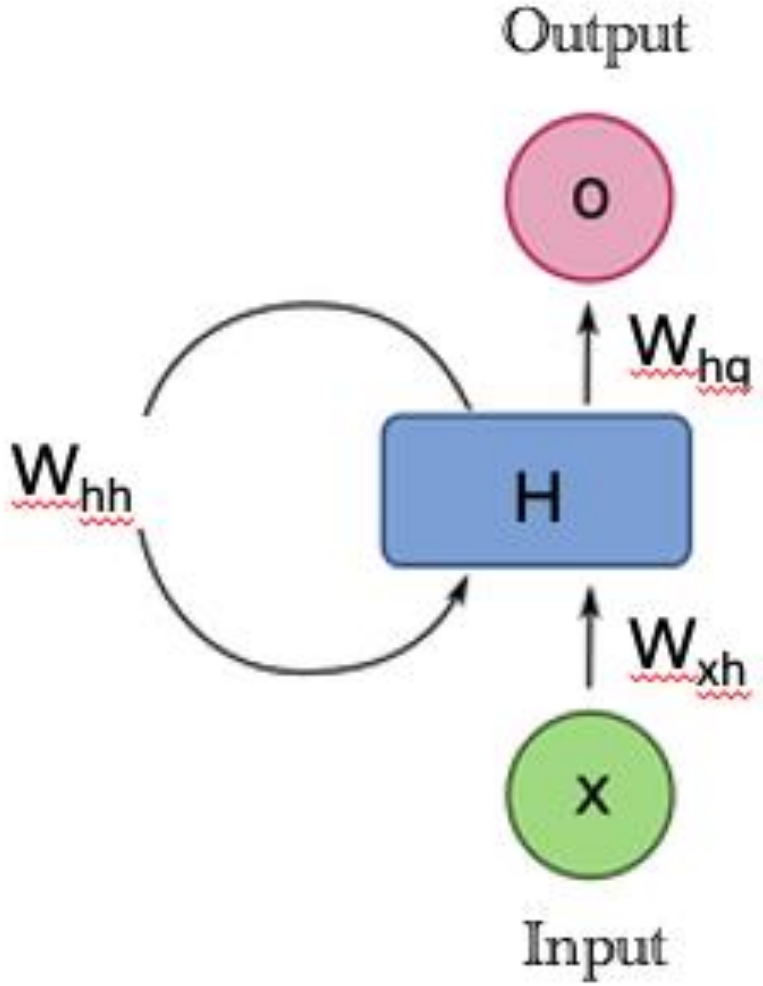
2014: Self attention for machine translation. Badanhau et al.

2017: Transformers. Vaswani et al. Attention Is All You Need.

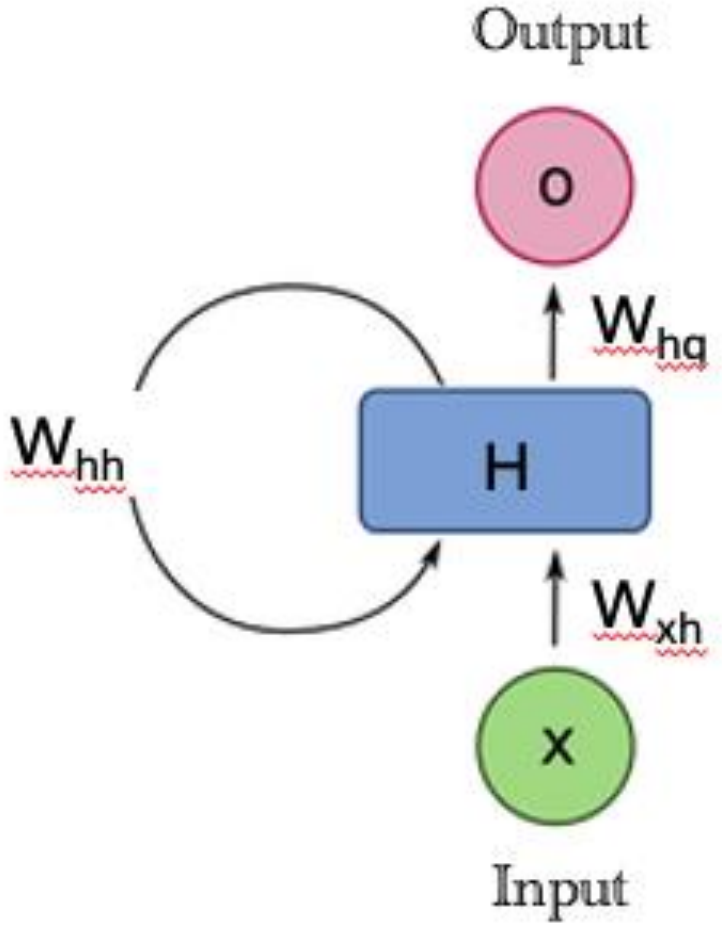


RNN: The basic architecture

- FFNN process all input features simultaneously.
- The idea of RNN is to process sequence of data,
- And through an internal memory (Hidden state) save relevant information to persist across time steps



RNN: Defining the architecture



$$O \in \mathbb{R}^{1\times d}$$

It might depend on the **task**, but for text prdiction/generation, it will have the same size as input

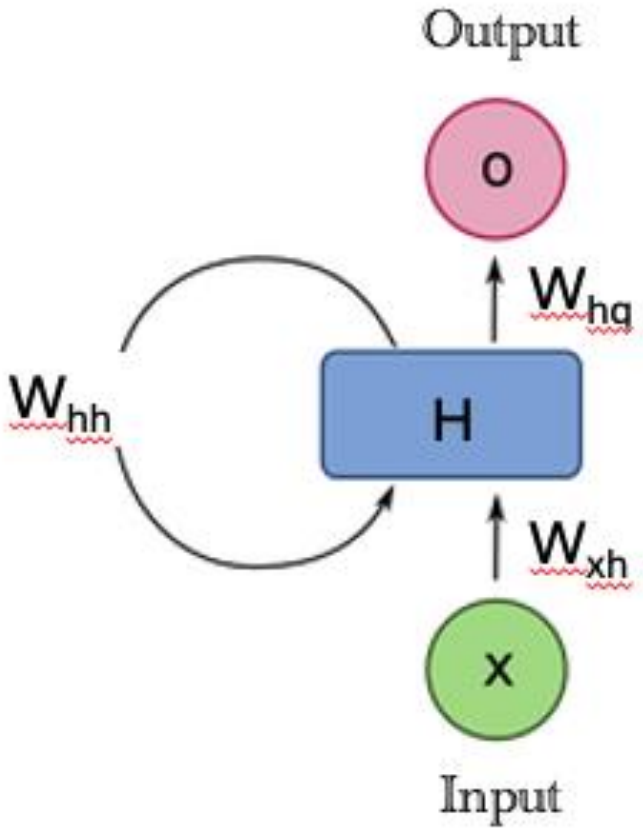
$$H \in \mathbb{R}^{1\times h}$$

Current state of the Hidden Units. Number of Hidden units is an hyperparameter to be determined.

$$X_t \in \mathbb{R}^{1\times d}$$

Vector representation of characters (or words).
Size determined by the size of the vocabulary used

RNN: Weight matrices

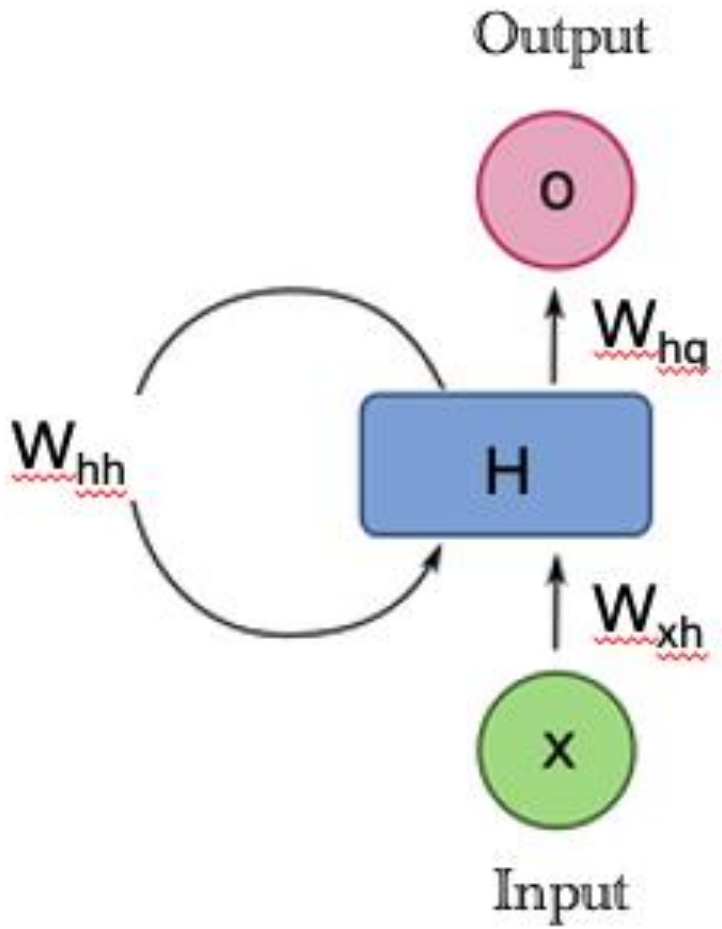


W_{hq} Hidden to Output $\boldsymbol{W}_{hq} \in \mathbb{R}^{h \times d}$

W_{hh} Hidden to Hidden units $\boldsymbol{W}_{hh} \in \mathbb{R}^{h \times h}$

W_{xh} Input to Hidden units $\boldsymbol{W}_{xh} \in \mathbb{R}^{d \times h}$

RNN: Forward pass

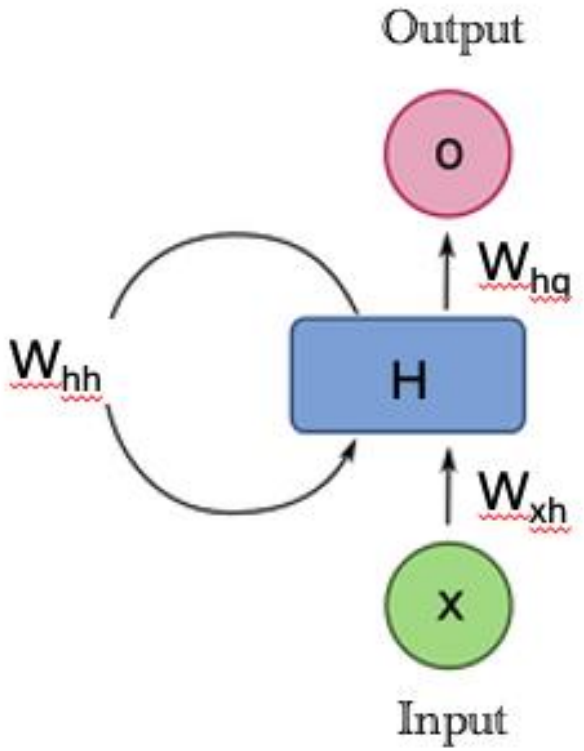


$$\mathbf{O}_t = \mathbf{H}_t \mathbf{W}_{\text{hq}} + \mathbf{b}_{\text{q}}.$$

$$\mathbf{H}_t = \phi(\mathbf{X}_t \mathbf{W}_{\text{xh}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{hh}} + \mathbf{b}_{\text{h}}).$$

$$X_t \in \mathbb{R}^{1 \times d}$$

RNN: Activation functions



$$\mathbf{H}_t = \phi(\mathbf{X}_t \mathbf{W}_{\text{xh}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{hh}} + \mathbf{b}_{\text{h}}).$$

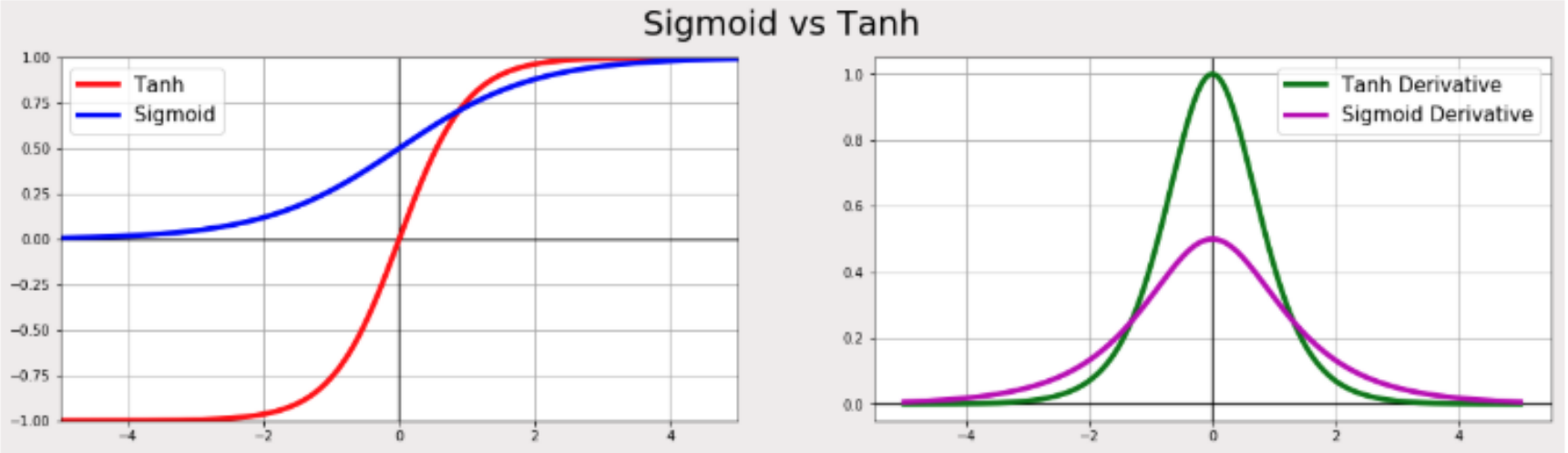
In the hidden layer the standard architecture uses a Hyperbolic tangent (outputs $[-1,1]$). The Sigmoid suffers more of the vanishing gradient problem. ReLu are a more recent valid alternative.

$$\mathbf{O}_t = \mathbf{H}_t \mathbf{W}_{\text{hq}} + \mathbf{b}_{\text{q}}.$$

The output as such is linear but an activation function can be used, and it would depend on the task.
Softmax function is a classic option for character prediction

RNN: Activation functions in the hidden layer

In the hidden layer the standard architecture uses a Hyperbolic tangent (outputs [-1,1]). The Sigmoid suffers more of the vanishing gradient problem. ReLu are a more recent valid alternative.



$$tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \qquad \sigma(x) = \frac{1}{1 + e^{-x}}$$

RNN: The Softmax function

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

For the case of having several posible outputs, as it will be the case of using text codified sequences, the softmax is very much used, converting a vector of raw, real-valued scores into a probability distribution over multiple, mutually exclusive classes. These probabilities sum up to 1, providing an interpretable measure of the model's confidence for each class.

Example:
Raw Output (z) = [2.1, 1.0, 0.5] (for A, B, C)

1. **Exponentiate each logit:**
   - $e^{2.1} \approx 8.166$
   - $e^{1.0} \approx 2.718$
   - $e^{0.5} \approx 1.649$
2. **Calculate the sum of exponentials (denominator):**
   - Sum $\approx 8.166 + 2.718 + 1.649 \approx 12.533$
3. **Divide each exponentiated value by the sum to get probabilities:**
   - **Probability of A:** $8.166/12.533 \approx 0.651$
   - **Probability of B:** $2.718/12.533 \approx 0.217$
   - **Probability of C:** $1.649/12.533 \approx 0.132$

Softmax Output = [0.651, 0.217, 0.132]

The model predicts:

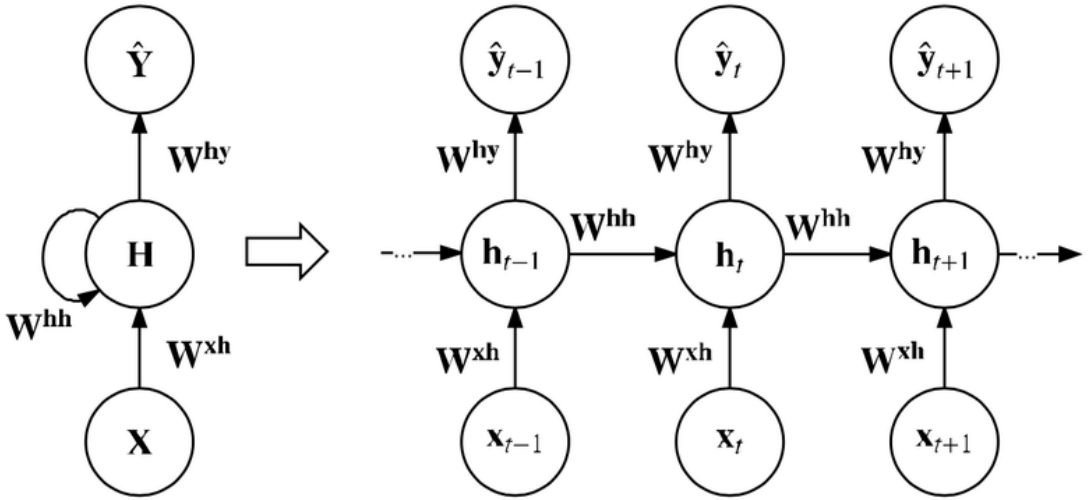
- A **65.1% chance** the classification is **A**.
- A **21.7% chance** the classification is **B**.
- A **13.2% chance** the classification is **C**.

The highest probability (0.651) determines the model's final prediction: **A**.

RNN: unfolding for time steps

IMPORTANT

We input one token at a time (in our case one character in the one-hot encoding scheme)

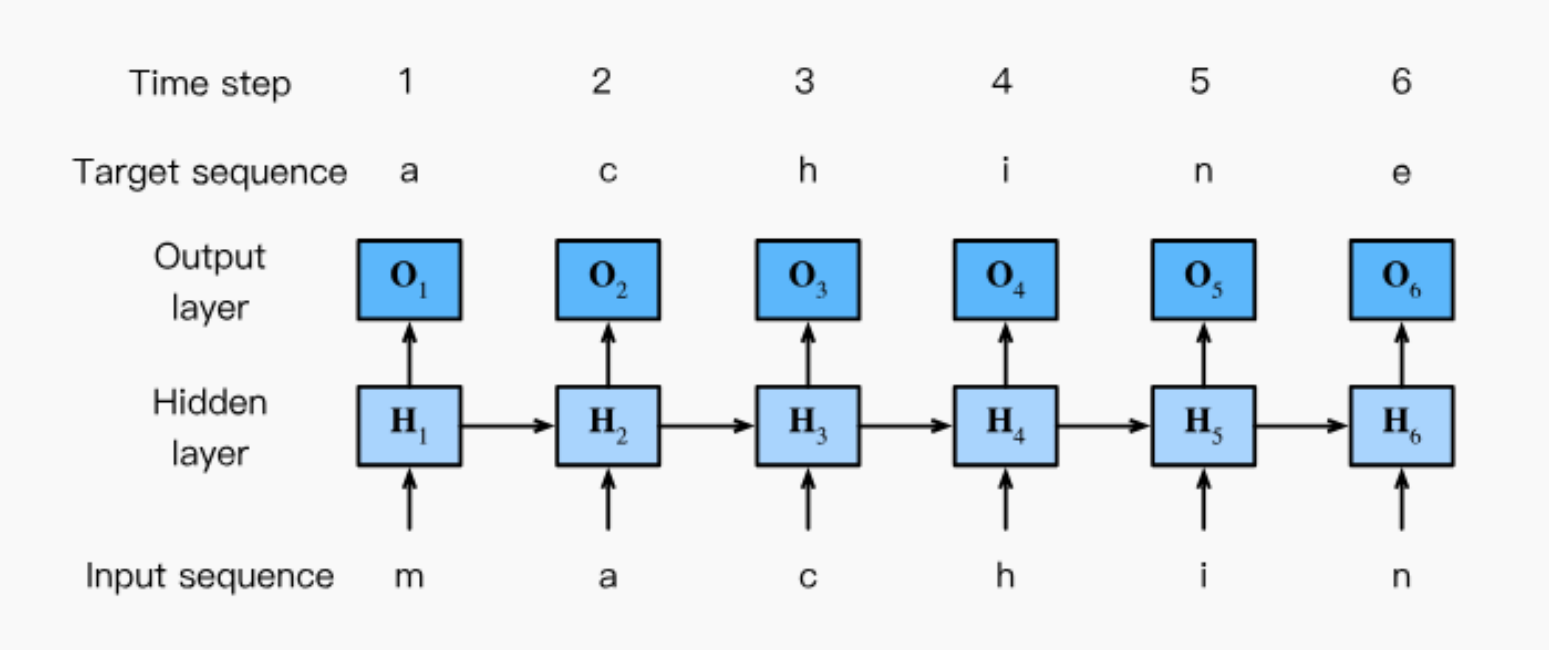


We unfold the network, and now looks like a FFNN, but the weight matrices are shared across time steps, and the algorithm of BPTT can be applied.

We have three weight matrices that are **shared** across all time steps:
At time step 0 we input $X_{t=1}$, at time step t, the input is Xt and so on

RNN: Sequence length: input = outputs

The time step (sequence length) is a critical **hyperparameter** that must be carefully selected based on the nature of the **problem**, the **data characteristics**, and the **model's inherent capabilities**



The sequence length balances the desired temporal context against the inherent limitations of the vanilla RNN architecture. Setting it too long results in training instability and a failure to learn distant information, while setting it too short limits the model's contextual understanding to only immediate inputs.

Inputs and outputs for 6 time steps:

INPUT : MACHIN

TARGET : ACHINE

Training RNN: BackPropagation through Time (BPTT)

Applying backpropagation in RNNs is called *backpropagation through time*.

The unrolled RNN is essentially a feedforward neural network with the special property that the same parameters are repeated throughout the unrolled network, appearing at each time step.

Then, just as in any feedforward neural network, we can apply the chain rule, backpropagating gradients through the unrolled net. The gradient with respect to each parameter must be summed across all places that the parameter occurs in the unrolled net.

We start with a simplified model of how an RNN works. This model ignores details about the specifics of the hidden state and how it is updated. The mathematical notation here does not explicitly distinguish scalars, vectors, and matrices. We are just trying to develop some intuition. In this simplified model, we denote h_t as the hidden state, x_t as input, and o_t as output at time step .

Training RNN: BackPropagation through Time (BPTT)

In this simplified model, w_h Includes matrix W_{xh} and W_{hh} simplified

$$h_t = f(x_t, h_{t-1}, w_h),$$
$$o_t = g(h_t, w_o),$$

where f and g are transformations of the hidden layer and the output layer, respectively. Hence, we have a chain of values that depend on each other via recurrent computation. The forward propagation is fairly straighforward. All we need is to loop through the triples one time step at a time. The discrepancy between output and the desired target is then evaluated by an objective function across all the time steps as

$$L(x_1, \ldots, x_T, y_1, \ldots, y_T, w_h, w_o) = \frac{1}{T}\sum_{t=1}^{T} l(y_t, o_t).$$

Training RNN: BackPropagation through Time (BPTT)

To compute the modification of weights, we compute the gradient of the Total Loss L:

$$\frac{\partial L}{\partial w_{\mathrm{h}}} = \frac{1}{T}\sum_{t=1}^{T}\frac{\partial l(y_t, o_t)}{\partial w_{\mathrm{h}}}$$
$$= \frac{1}{T}\sum_{t=1}^{T}\frac{\partial l(y_t, o_t)}{\partial o_t}\frac{\partial g(h_t, w_{\mathrm{o}})}{\partial h_t}\frac{\partial h_t}{\partial w_{\mathrm{h}}}.$$

The first and the second factors of the product are easy to compute. The third factor is where things get more complex.

With some mathematical working we can arrive to:

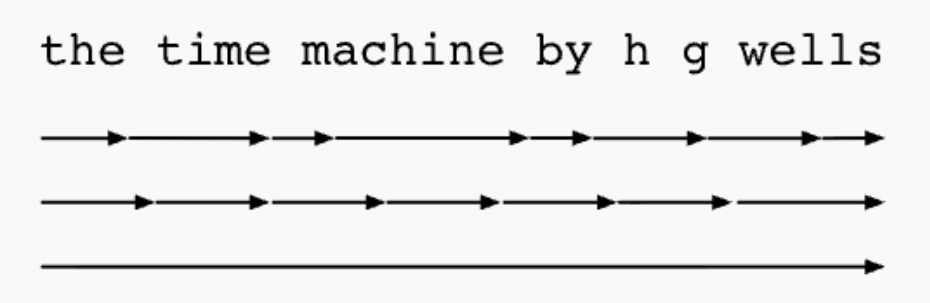
$$\frac{\partial h_t}{\partial w_{\mathrm{h}}} = \frac{\partial f(x_t, h_{t-1}, w_{\mathrm{h}})}{\partial w_{\mathrm{h}}} + \sum_{i=1}^{t-1}\left(\prod_{j=i+1}^{t}\frac{\partial f(x_j, h_{j-1}, w_{\mathrm{h}})}{\partial h_{j-1}}\right)\frac{\partial f(x_i, h_{i-1}, w_{\mathrm{h}})}{\partial w_{\mathrm{h}}}.$$

$$\frac{\partial h_t}{\partial w_{\mathrm{h}}} = \frac{\partial f(x_t, h_{t-1}, w_{\mathrm{h}})}{\partial w_{\mathrm{h}}} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^{t} \frac{\partial f(x_j, h_{j-1}, w_{\mathrm{h}})}{\partial h_{j-1}} \right) \frac{\partial f(x_i, h_{i-1}, w_{\mathrm{h}})}{\partial w_{\mathrm{h}}}.$$

For large sequences, the computation of the gradient involves several products and can lead to vanishing or exploding gradients, and thus some strategies should be applied in order to get good generalization:

Two main strategies have been successfully developed: **Clipping gradient and Truncation**

Truncated BPTT: modifications for avoiding vanishing gradients



Truncated BPTT divides the input text into short segments (typically 16–128 time steps), performs backpropagation only within each segment, and detaches the hidden state before processing the next segment.

Sequence length = how far gradients propagate through time.

Batch size = how many independent sequences contribute to each weight update.

Why not full BPTT?

Book length = 170,000 chars
Hidden size = 128

Full BPTT would unroll the network over all 170,000 time steps before computing gradients.

Massive memory usage (need to store all hidden states)

Extremely slow backward pass
Severe vanishing/exploding gradients.

BPTT modifications for avoiding exploding gradient: Clipping

When we say that gradients **explode**, we mean that their values become excessively large during backpropagation. As a result, the corresponding weight updates can be extremely large, causing the training process to become unstable and potentially diverge instead of minimizing the objective function. One possible solution is to use a very small learning rate, but this may significantly slow down training. A more effective approach is to apply **gradient clipping**, a heuristic that limits the magnitude of the gradients by projecting them onto a ball of a predefined radius whenever their norm exceeds a given threshold.

$$\mathbf{g} \leftarrow \min\left(1, \frac{\theta}{\|\mathbf{g}\|}\right)\mathbf{g}.$$

This ensures that the gradient norm never exceeds θ and that the updated gradient is entirely aligned with the original direction of $\boldsymbol{g}$. It also has the desirable side-effect of limiting the influence any given minibatch can exert on the parameter vector. In any case is a heuristic modification that usually works.

RNN: Successful Experiments

The main success of vanilla RNNs in text prediction was proving the *concept* that a neural network could leverage temporal context but success was limites to small scale scenarios due to architectural limitations:

- **Modeling Cognitive Processes**: In the early 1990s, Paul Elman's experiments used SRNs to study how networks could acquire an understanding of grammatical structure and predict the next word or character in simplified language streams. These experiments were successful in showing that the networks could learn internal representations for sequence data, something traditional feedforward networks could not do.

- **Character-Level Language Modeling (Short Sequences)**: On small corpora of text, like a few pages of a novel or simple poetry, vanilla RNNs could be trained to generate text by predicting the next character. The resulting text often had correct local syntax (e.g., forming proper English words, correct use of quotes and spacing) but quickly lost coherence over longer sequences, as the network struggled to maintain a consistent high-level context or topic.

RNN: Putting all to work and further improvements

Consider these improvements when optimizing neural networks:

- **Monitor learning curves and performance metrics** to understand model behavior and improve optimization.

- **Choose the right architecture:** Analyze network performance using different hidden layer sizes, as architecture affects performance and efficiency.

- **Optimize epochs and efficiency:** Use training, validation, and test sets with an early stopping scheme and a "patience" value. This method optimizes results and reduces training time.

- **Address poor learning:** If training does not improve with larger architectures, the network may not be learning effectively. Consider changing activation functions (e.g., from Hyperbolic Tangent to ReLU) and applying dropout to reduce overfitting.

- **Weight Initialization.** Consider different options (see next slide)

RNN: Weight initialization

Small Random Values (Gaussian or Uniform): The standard baseline where weights are drawn from a random distribution with a small range (e.g., [-0.1,0.1]). This simple method breaks symmetry, ensuring different neurons learn unique features and preventing immediate saturation of activation functions like tanh. It is simple to implement but may not be the most efficient for deep temporal dependencies.

Xavier/Glorot Initialization: This approach scales random weights based on the number of input and output units of the layer to maintain consistent variance of signals flowing through the network. It significantly mitigates vanishing/exploding gradients in shallower networks and is well-suited for symmetric activation functions like tanh and sigmoid, leading to faster and more stable convergence.

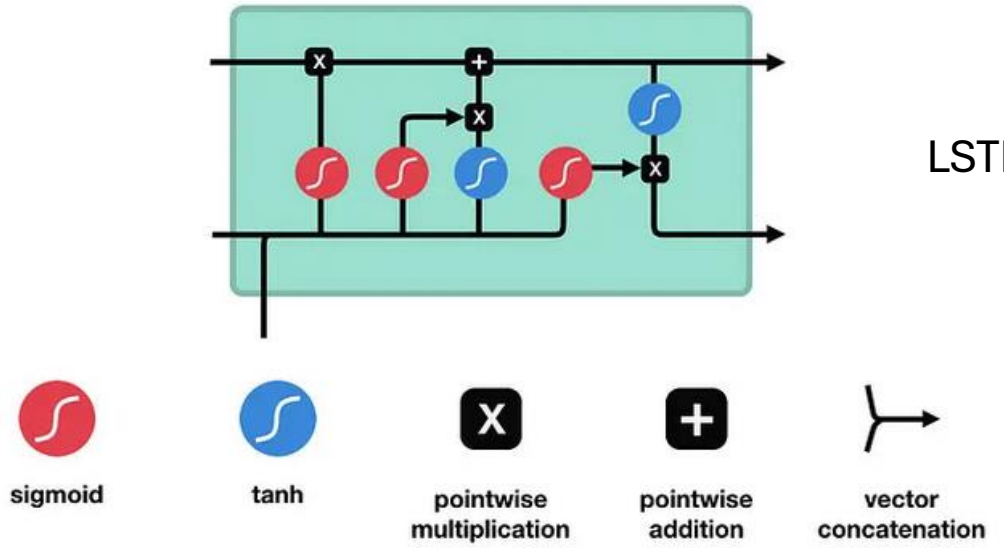
Orthogonal Initialization: Weights (especially the recurrent weight matrix) are initialized as random orthogonal matrices, which helps preserve the norm of the gradients during backpropagation through time (BPTT). This method specifically targets the RNN gradient problem, allowing information and gradients from the far past to flow more effectively, enhancing the network's ability to learn long-term dependencies.

He Initialization (for ReLU activation): If using ReLU activations, He initialization is preferred, scaling random weights based only on the number of input connections. The higher variance helps prevent the "dying ReLU" problem by ensuring neurons remain active and pass effective gradients, although gradient clipping may still be necessary for stability with ReLU in vanilla RNNs.

RNN: Limitations and evolution.

The "successful" experiments using vanilla RNNs were always constrained by the **vanishing gradient problem**, which made it nearly impossible for the network to remember information from more than a few time steps prior. As a result, they were **not successful in real-world, large-scale NLP applications** where understanding long-term dependencies (e.g., an entire paragraph's topic or an initial statement made at the start of a long sentence) is critical.

In essence, while they achieved conceptual success in research, they were quickly superseded by LSTMs and GRUs for practical text prediction tasks.

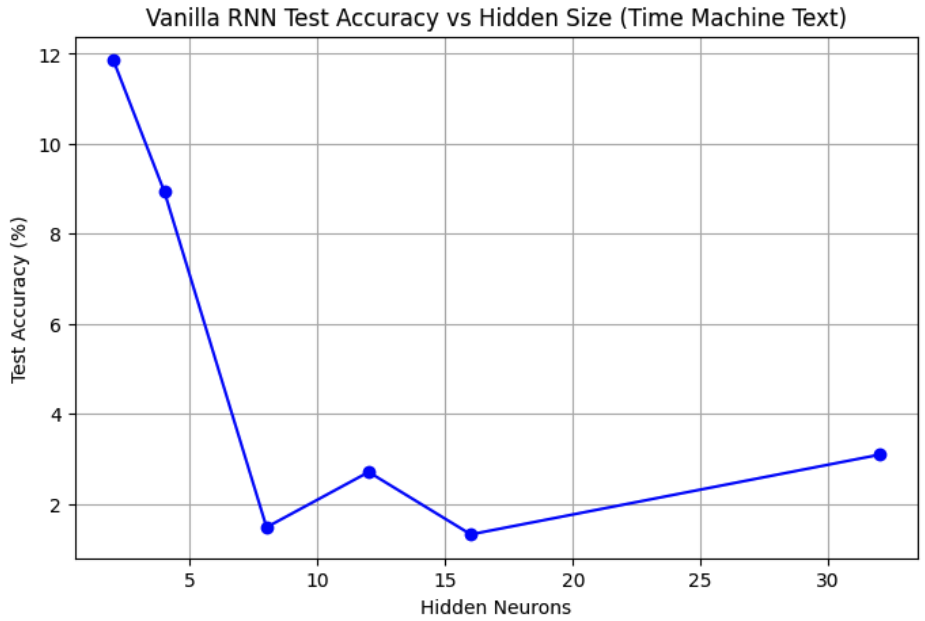


LSTM CELL

RNN – P2-Lab2 – **Two tasks**

- Task 1: Predicting names.

- Task 2: Optimize a Vanilla RNN in next character prediction task using the text of the book "The Time Machine".



Task 1: *Lab2_RNN_PredictingNames.ipynb*

Training a RNN for the task of determine the origin of names from 18 different countries.

Input: one character at a time is presented in one-hot -encoding scheme size 58 (vocabulary), time Length is set internally according to the length of each name, and the output is the codification for the corresponding Country.

Hidden layer size is an hyperparameter set initially to 128.

Output in this case is a vector of dimension 18, corresponding to the posible countries.

Analyze the code, understand it, execute and improve some parts. Draw some conclusions from the results obtained, in particular from the confussion matrix.

Do you think a recurrent neural network perform much better for this problem
than a feedforward network? Run a code with similar parameters and verify.

Task 2 – Next Character Prediction

Optimize a Vanilla RNN for the next character prediction task using the text of the book "The Time Machine".

Using one-hot-encoding (external embedding not permitted) and using the standard RNN architecture, you should optimize:

Learning Rate, Optimization Algorithm, Activation functions, Improvements for avoiding Overfitting/vanishing-exploding Gradients, Weight initialization.

Training with 30% initial part of the test, next 10% for Validation and remaining 60% for Test. Max. Sequence length =20.

I will provide initial code and then you have to modify it until finding your best solution.

When you think you have the hyperparameters set, you should rexecute a final experiment with 20 independent runs (initialization changes on every one of them) and report average learning curve with standard deviation and final accuracy and std deviation (.ipynb notebook should be submitted).

Robustness and Smoothness

Robustness refers to the model's ability to maintain stable and accurate performance even when presented with slight variations, noise, or minor distortions in the input data.

When parameters are well-optimized, they have converged to a state where the model doesn't rely too heavily on any single, specific feature or exact input value.

Smoothness refers to the nature of the function the neural network learns. A smooth function means that small changes in the input result in only small, proportional changes in the output. There are no abrupt jumps or cliffs in the decision boundaries.

In summary: Smoothness is a characteristic of *how* the model maps inputs to outputs, while robustness is a characteristic of *how well* the model performs under typical real-world imperfections. You strive for smoothness during training to ensure robustness in deployment.

NEXT CHARACTER PREDICTION: Advance recommended reading

Karpathy, A. (2015). *The Unreasonable Effectiveness of Recurrent Neural Networks*. Andrej Karpathy Blog. Disponible en: https://karpathy.github.io/2015/05/21/rnn-effectiveness/

Al-Rfou, R., Choe, D., Constant, N., Guo, M. y Jones, L. (2018). *Character-Level Languague Modeling with Deeper Self-Attention*. (link a pdf)